The C++ Compendium — Volume I

Guiding Principles, the C++26 Baseline, AI, Sound, and Cross-Device Interfacing

*Compiled August 2026. Current standard: **ISO C++26**, technical work completed 28 March 2026 (London/Croydon), replacing C++23 (ISO/IEC 14882:2024).*

Part 0 — Charter

This volume establishes the **guiding principles** that govern all C++ work in this body of knowledge. Everything downstream — libraries, platforms, build systems, AI integration, audio engines — is an

application of the principles in Part 1, constrained by the standard baseline in Part 2.

Three reading rules:

1. **Principles outrank idioms.** An idiom that violates a principle is a legacy artifact, not a style choice.
2. **The standard is the contract; the ABI is the reality.** Portable source does not imply portable binaries. Part 6 is not optional reading.
3. **Cite the version.** "C++" is not a language; C++11, C++17, C++20, C++23, and C++26 are. Every rule here is tagged with the standard it assumes.

Part 1 — The Foundational Principles

P1. Zero-overhead abstraction

You don't pay for what you don't use, and what you do use, you couldn't hand-code better.

This remains the load-bearing principle of the language and it survived the C++26 safety push intact. Safety features added in C++26 are either free at runtime or explicitly opt-outable in a hot path. Abstraction that costs runtime for no semantic gain is a defect, not a trade-off.

Corollary: measure before you assert. "Zero-overhead" is a claim about generated code, verifiable in a disassembler, not a claim about aesthetics.

P2. RAII and deterministic lifetime

Every resource — memory, file descriptor, socket, mutex, GPU buffer, audio stream, model handle — is owned by an object whose destructor releases it. There is no second mechanism. Garbage collection is

not available and finalizers are not a substitute.

- Raw `new`/`delete` in application code is a defect. Use `std::make_unique`, `std::make_shared`, or a container.
- Raw pointers are permitted as **non-owning observers only**.
- Lifetime bugs are the dominant source of C++ CVEs. Everything in Part 7 exists to catch them.

P3. Value semantics by default, reference semantics by exception

Prefer types that copy, compare, and move like integers. Reach for indirection only when identity, polymorphism, or size demands it. Move semantics make this cheap; C++26's reflection makes it mechanizable.

P4. Type safety and the expressive type system

Encode invariants in types, not comments. A `Duration` is not an `int`. A `SampleRate` is not a `size_t`. Strong typedefs, `enum class`, concepts (C++20), and the in-flight quantities-and-units library (P3045, targeting C++29) all serve this principle.

P5. Safety by default, opt-out by intent — *new in C++26*

This is the principle that changed. C++26 shifts the default posture of the language from "fast unless you ask for safety" to "safe unless you ask for speed."

Two mechanisms deliver it, and both apply **just by recompiling existing code**:

- **Erroneous behavior for uninitialized reads.** Reading an uninitialized local variable is no longer undefined behavior.

An entire vulnerability class is removed by a `-std=c++26` flag flip.

- **The hardened standard library.** Bounds checking on dozens of the most-used operations on `vector`, `span`, `string`, `string_view`, and more, standardized cross-platform.

The empirical case is unusually strong for a language feature. Per the November 2025 ACM Queue report on the pre-standard rollout at Google: across hundreds of millions of lines of C++, only five services opted out entirely, the fine-grained unsafe-access API was used in **seven** places total, average overhead was **0.3%**, over 1,000 bugs were fixed, and the production-fleet segfault rate dropped **30%**.

Policy: hardening is on. Opting out is a reviewed, justified, localized decision with a benchmark attached.

P6. Contracts express intent, not error handling — *new in C++26*

C++26 adds language-level `pre`, `post`, and `contract_assert`. These state what a function requires and guarantees. They are not a replacement for exceptions or `std::expected`.

Be aware this feature shipped over sustained dissent: the final plenary vote was 114 in favor, 12 opposed, 3 abstaining, with Bjarne Stroustrup among the objectors. Adopt contracts for functional-safety-relevant interfaces first; do not retrofit the whole codebase in one pass.

P7. Structured concurrency

Concurrency has lifetime, and that lifetime nests. C++26's `std::execution` (senders/receivers) is the standard expression of this: it makes structured, data-race-free-by-construction concurrency

the default shape rather than a discipline you impose by hand.

Practical warning from the committee itself: `std::execution` is production-viable but currently under-documented and short on ecosystem adapter libraries. Budget learning time and expect to write glue.

P8. Portability is a build-system property, not a source property

Standard-conforming source is necessary and nowhere near sufficient. ABI, libc, page size, ISA baseline, and toolchain all cross the boundary. See Part 6.

P9. Composition over frameworks at the seams

Own your `main()`, your allocation strategy, and your threading model. Frameworks (JUCE, Qt, Unreal) are legitimate inside their

domain; they should not dictate the shape of your core logic.

P10. Verify mechanically

Human review does not catch lifetime bugs at scale. Sanitizers, fuzzers, static analysis, and hardened builds in CI are not optional infrastructure — they are how P2 and P5 are actually enforced.

Part 2 — The Standard Baseline: C++26

C++26 is described by Herb Sutter as **the most compelling release since C++11**. That framing is worth taking seriously for planning purposes: C++14 through C++23 delivered features that mattered enormously to subsets of developers (parallel STL, concepts, coroutines,

modules). C++26 delivers features that affect essentially every C++ developer.

The four headline features

Feature	What it is	Why it r
Reflection	Compile-time introspection and code generation; C++ can describe itself and generate more	The larg upgrade templat Serializ ORM, bi generat to-string metapr stop be and-hac discipli
Less UB / hardened stdlib	Uninitialized reads become erroneous, not undefined; bounds-checked std ops	Memory improve **no code** — recor

Feature	What it is	Why it r
Contracts	`pre`, `post`, `contract_assert`	Functio strictly C's `ass`
`std::execution`	Unified sender/receiver async and parallelism framework	One mo CPU thr thread p GPU off structur concurr

Other adopted material worth knowing

- **Parameter pack indexing** (P2662) — eliminates recursive template metaprogramming for a very common case
- `constexpr` **containers and adaptors** (P3372) — much more of the STL usable at compile time

- `std::simd` — portable data-parallel types, relevant to both Part 4 and Part 5
- **Reduced UB across the board**, continuing into C++29

Compiler reality as of mid-2026

- **GCC 16.1** (April 2026) supports most C++26 features; reflection and contracts are merged in trunk.
- **Clang/LLVM** tracked at roughly two-thirds of C++26 throughout development.
- **MSVC** lags on the marquee features; verify before committing on Windows-first projects.

Sutter's prediction — and it is a reasonable planning assumption — is that **C++26 adoption will be materially faster than C++17/20/23**, because demand is high and implementations arrived early.

What comes next: C++29

Work began June 2026 (Brno). The dominant theme is **more memory safety**:

- Stroustrup's **P3984 type safety profile**, built on Gabriel Dos Reis's profiles framework (SG23)
- Further UB reduction proposals headed to EWG
- **P4158R0** (Oliver Hunt, Apple) — the WebKit experience report: ~4M lines of C++ hardened via a subset-of-superset approach, closing multiple vulnerability classes and covering the majority of historical exploits
- **P3045** quantities and units library, advanced to LEWG

Standard-adoption policy (recommended)

Project type	Target
New greenfield, controlled toolchain	**C++26** — take reflection and hardening immediately
Existing codebase, active development	**C++26 compile flag first** (free safety), features incrementally
Android NDK / mobile	**C++20 baseline, C++26 where NDK Clang allows** — set `CMAKE_CXX_STANDARD` explicitly; CMake defaults to Clang's default, historically C++14
Embedded / certified / vendor-locked toolchain	**C++17** floor, plan the C++26 jump around the safety story

Part 3 — Codified Rules

These are the enforceable rules. Each maps to a principle.

Lifetime and memory

#	Rule	Principle
L1	No owning raw pointers. `unique_ptr` by default, `shared_ptr` only for genuine shared ownership	P2
L2	No manual `new`/`delete` outside of allocator/container implementations	P2
L3	Never return a reference or pointer to a local	P2
L4	Prefer `std::span` / `std::string_view` for non-owning views; never outlive the owner	P2, P5

#	Rule	Principle
L5	Rule of Zero. If you write one of the five, justify why the Rule of Zero fails	P2, P3

Type and interface

#	Rule	Principle
T1	`enum class` over unscoped enums, always	P4
T2	Constrain templates with concepts; do not rely on SFINAE for new code	P4
T3	`explicit` on single-argument constructors unless implicit conversion is the point	P4
T4	Express preconditions as `pre`, not as comments or defensive `if`	P6

#	Rule	Principle
T5	Errors: `std::expected` for expected failures, exceptions for exceptional ones, contracts for programmer errors. Do not mix the three for one condition	P6

Concurrency

#	Rule	Principle
C1	Shared mutable state requires a mutex or an atomic; there is no third option	P7
C2	Prefer `std::execution` / `std::jthread` over raw `std::thread`	P7
C3	Lock ordering is documented and total, or you will deadlock	P7

#	Rule	Principle
C4	Never block, allocate, or lock on a real-time thread — see Part 5	P1, P7

Build and boundary

#	Rule	Principle
B1	Every cross-binary boundary is a **C ABI** boundary unless both sides are built by the identical toolchain	P8
B2	Never pass `std::string`, `std::vector`, or any standard type across a shared-library boundary between different compilers or standard-library versions	P8
B3	Warnings-as-errors in CI: `-Wall -Wextra -Wpedantic -Werror` / `/W4 /WX`	P10

#	Rule	Principle
B4	Sanitizer builds (ASan, UBSan, TSan) run on every PR	P10
B5	Pin toolchain versions in the build system; a floating compiler is an unreproducible build	P8

Part 4 — AI and C++

C++ occupies two distinct positions in the AI stack, and conflating them causes bad architecture decisions.

4.1 C++ as the substrate

Nearly every production inference engine is C++ underneath a Python skin. This is not incidental — inference is a latency, memory-bandwidth, and hardware-dispatch problem, which is exactly what P1 exists for.

Library	Written in	Best for
ONNX Runtime	C++ (APIs in C++, Python, C#, Java, JS, Julia, Ruby)	Portable, well-bounded inference a stable exported model; runs on Linu macOS, Windows, iOS, Android
LibTorch	C++	When your app itsel must speak in tensors, modules, and framework semantics
llama.cpp / GGML	Plain C/C++, MIT, no dependencies	Local quantized LL inference in a native app; CPU-first, GPU-capable (CUDA, Metal, HIP/ROCm)
oneDNN / OpenVINO	C++	CPU efficiency, grap optimization, Intel-heavy deployment

Library	Written in	Best for
TensorFlow Lite	C++	Small, offline, batte sensitive, embedde targets
TensorRT-LLM / TurboMind (LMDeploy)	C++	Maximum NVIDIA throughput; TurboMind is a pure C++ engine
Triton Inference Server	C++	Orchestration/contr plane hosting other runtimes
OpenCV (`cv::dnn`)	C++	Vision pipelines where you already need image I/O

The selection principle: start from the product, not the tool. Ask what your application *owns* versus what it merely *consumes*. If it consumes a frozen model → ONNX Runtime. If it owns tensor semantics → LibTorch. If it owns a local

quantized LLM → llama.cpp. If it owns nothing and orchestrates everything → Triton.

Then ask the second question: **where will the engineering pain actually be paid?** Export friction, binary size, quantization quality, and hardware-backend maintenance are the four usual answers, and they are not interchangeable.

4.2 Principles for embedding inference in a C++ application

- **AI-1 — The model is a resource (P2).** Wrap the session/context handle in an RAII type. `Ort::Session`, `llama_context`, `torch::jit::Module` all get owners with destructors. No global model singletons.
- **AI-2 — Inference is not real-time.** Never call an inference API from an audio callback, a render thread, or an interrupt

handler. Cross the boundary with a lock-free queue. See Part 5.

- **AI-3 — Load once, infer many.** Model load is orders of magnitude more expensive than a forward pass. Structure lifetime accordingly.
- **AI-4 — Quantization is a product decision, not an optimization.** 1.5-bit through 8-bit integer paths and KV-cache quantization (int8/int4, AWQ, MXFP4) change output quality, not just speed. Measure quality, not only latency.
- **AI-5 — Pin the runtime version.** Inference libraries break ABI and file formats between minor releases far more often than the C++ ecosystem norm.
- **AI-6 — Isolate the backend behind your own interface.** A `class Inferencer` with a C-ABI-safe boundary lets you swap ONNX Runtime for llama.cpp without

touching application code. This is P9 applied.

- **AI-7 — Treat vendor benchmarks as vendor claims.** Throughput headlines are self-reported until you reproduce them on your hardware with your model.

4.3 AI as a tool for writing C++

The third axis, and the one most likely to be underweighted. Agentic coding tools now produce large volumes of C++, which shifts where defects come from.

- **AI-8 — AI-generated C++ inherits none of your invariants.** It will produce plausible code that violates P2 and P5 in ways that compile cleanly. Sanitizer and static-analysis gates matter *more* in an AI-assisted workflow, not less.
- **AI-9 — Review the lifetime, not the syntax.** Generated code is usually syntactically excellent. Ownership,

aliasing, and thread-affinity errors are where it fails.

- **AI-10 — Reflection reduces the need for generated boilerplate.** Much of what AI assistants are currently asked to generate — serializers, enum-to-string, binding shims — becomes a compile-time library problem in C++26. Prefer the language mechanism over the generated artifact: it cannot drift.
- **AI-11 — C++26 hardening is a safety net for machine-written code.** The 0.3%-overhead bounds checking is worth strictly more when a nontrivial fraction of your code was not written by a human who reasoned about the bounds.

Part 5 — Sound and Audio

> **Terminology note:** "C sound" is read here in both senses — *sound programming in*

> *C++* generally, and **Csound** specifically, the sound-and-music computing system (originally MIT, Barry Vercoe, 1984; named for being the first C-language system of its type). Both are covered; §5.4 is Csound proper.

5.1 The real-time audio constraint — the hardest rule in this compendium

C++ dominates professional audio because audio has a hard deadline: a buffer of samples must be produced before the DAC needs it, every time, or the user hears a click. A 128-frame buffer at 48 kHz gives you **2.67 milliseconds**, with no exceptions and no retries.

Inside an audio callback, the following are forbidden:

Forbidden	Why
Heap allocation / deallocation	The allocator may take a global lock or a page fault of unbounded duration
Mutex lock	Priority inversion; the OS scheduler is not on your deadline
Any system call (file, network, logging)	Unbounded latency
`std::string`, `std::vector`, `std::map` operations that may allocate	Same as above
Exceptions (throwing)	Unbounded unwinding cost
`dynamic_cast` / RTTI in hot paths	Runtime lookup cost
Inference calls, GC'd runtimes, JNI	Categorically unbounded

Forbidden	Why
transitions	

The permitted communication mechanism between the audio thread and everything else is a **lock-free single-producer/single-consumer queue** or an atomic with relaxed/acquire-release ordering, with all memory pre-allocated at stream setup.

This is P1 and P7 taken to their limit. It is also the clearest case in the language where "safe by default, opt out with intent" (P5) needs a documented, benchmarked opt-out.

5.2 The audio stack, by layer

Layer 1 — Platform I/O (the OS talks here)

Platform	Native API
Linux	ALSA (kernel-adjacent), PipeWire (modern default), JACK (pro-audio)

Platform	Native API
Windows	WASAPI (modern), ASIO (pro, third-party)
macOS / iOS	Core Audio / AudioUnit
Android	**AAudio** (API 27+), OpenSL ES (legacy)
Web	Web Audio API / WASM

Layer 2 — Portable I/O wrappers

Library	Character
miniaudio	Single source file, no dependencies, public domain, C and C++. The lowest-friction choice
RtAudio	Small, mature, C++-native
PortAudio	The long-standing C portability layer

Library	Character
Oboe	Android-specific C++ wrapper — see §5.5
SDL3 audio	If you already have SDL for windowing/input

Layer 3 — Frameworks

- **JUCE** — the industry standard C++ framework for audio applications and plugins. One codebase targets **VST, VST3, AU, AUv3, AAX, and LV2** plus standalone apps, across Windows, macOS, Linux, iOS, and Android. Integrates via CMake (preferred) or the Projucer generator. Includes DSP primitives, FFT, filters, oscillators, and a GUI toolkit.

Layer 4 — DSP and synthesis

- **STK** (Synthesis ToolKit) — C++, real-time synthesis and physical modeling, long academic pedigree

- **Soundpipe** — C, 100+ modular DSP units; excellent for composing signal chains
- **Essentia** — C++ analysis, description, and synthesis with Python bindings
- **FAUST** — a functional DSP language that *compiles to C++*; the right answer when the DSP algorithm is the product
- `std::simd` (C++26) — portable vectorization for your own kernels, finally standard

5.3 Audio rules

#	Rule	Principle
S1	Pre-allocate every buffer at stream open. The callback allocates nothing, ever	P1, C4
S2	Audio thread ↔ UI thread communication is lock-free only	P7
S3	Process in `float` (or `double`) internally; convert	P4

#	Rule	Principle
	at the boundaries	
S4	Parameterize sample rate and buffer size; never hardcode 44100 or 512	P4, P8
S5	Denormals kill performance — flush-to-zero on the audio thread	P1
S6	Test the callback under a sanitizer build *and* profile it; ASan changes timing, so do both separately	P10
S7	Latency is a measured number, not a design intention. Instrument round-trip latency and underrun counts	P10

5.4 Csound

Csound is a domain-specific language and engine for audio synthesis and music

computing, LGPL 2.1+, cross-platform (Linux, Windows, macOS, iOS, Android). **Csound 7.x is the current development line; the 6.x series is end-of-life with no further releases planned.** Note that Csound 7 removed several Csound 6 API functions (MIDI device selection, some instance control), so ports are not always mechanical.

Why it belongs in a C++ compendium: Csound is designed to be *embedded*. It is a synthesis engine you can host inside a C++ application, and it is extensible by C++ plugins you write.

The two directions of the interface:

(a) C++ as host. Include `csound.hpp`, link `libcsound`. The C API (`csound.h`) uses an opaque pointer representing a Csound instance, passed as the first argument to every call — an object model expressed in C. The C++ API wraps this in a class. Your

application drives the engine: compile an orchestra, send score events, read and write control channels during performance, and pull audio buffers. The `csound` command-line program is itself built on this API, which is a useful guarantee that the API is complete.

(b) C++ as plugin. Csound loads shared libraries at runtime that implement external opcodes or audio/MIDI drivers. Plugins include `csdl.h`. For C++ specifically there are two base paths, and the choice is an allocator decision:

- `include/plugin.h` — uses the **Csound allocator**, for opcodes that avoid standard-library collections
- `include/OpcodeBase.hpp` — uses the **standard C++ allocator**, for opcodes that do use STL containers

Csound rules:

#	Rule
CS1	Wrap the Csound instance in an RAII type. The opaque pointer is a resource (P2)
CS2	Choose the plugin base class by allocator requirement, not by familiarity
CS3	Target the Csound 7 API for new work; treat 6.x compatibility shims as migration aids, not architecture
CS4	Csound performance runs on a real-time thread. §5.1 applies in full inside custom opcodes
CS5	Communicate with a running performance through control channels, not by recompiling the orchestra

5.5 Audio on Android — where Parts 5 and 6 meet

Oboe is Google's open-source C++ library, part of the AGDK. It is a thin wrapper, and understanding exactly how thin is important: it selects **AAudio on API 27+ (Android 8.1)** and falls back to **OpenSL ES** on older devices, works back to API 16 (~99% of devices), and adds automatic latency tuning plus a `QuirksManager` that works around known device-specific audio bugs.

What Oboe is not: it performs no audio processing and does not intrinsically lower latency. It routes audio in and out and configures the underlying API well. The DSP is still yours.

For lowest latency:

```
builder.setPerformanceMode(oboe::PerformanceMode::LowLatency);
builder.setSharingMode(oboe::SharingMode::Exclusive);  // may be denied
```

Exclusive sharing mode, when granted, writes directly into the MMAP buffer read by the DSP. Monitor `AAudioStream_getXRunCount()` — underruns mean the buffer is too small. If you cannot use Oboe, use AAudio directly; both default to a *higher*-latency mode unless you explicitly request low latency.

Part 6 — Interfacing Across Devices: Linux, Android, x86-64, and Beyond

6.1 The four-layer portability model

Portability failures happen at a specific layer. Diagnose by layer.

Layer	What varies	Your tool
1. Language	Standard version, compiler extensions	`-std=c++26`, `-Wpedantic`, no compiler-specific extensions
2. ABI	Calling convention, name mangling, struct layout, exception tables, `std::` type layout	**C ABI at boundaries** (B1, B2)
3. System	libc, syscalls, filesystem, threading, audio/graphics APIs	Abstraction layer or platform `#if` at *one* seam

Layer	What varies	Your tool
4. Build	Toolchain, packaging, page size, alignment, signing	CMake + toolchain files + a real CI matrix

Most cross-device pain is Layer 2 or Layer 4, and most engineers debug it at Layer 1.

6.2 x86-64

There are **two incompatible x86-64 ABIs**, and this is the single most common source of "it works on Linux but not Windows."

	System V AMD64 ABI	Microsoft x64 ABI
Used by	Linux, macOS, BSD, Android x86_64	Windows
Integer arg registers	RDI, RSI, RDX, RCX, R8, R9	RCX, RDX, R8, R9

	System V AMD64 ABI	Microsoft x64 ABI
Float args	XMM0–XMM7	XMM0–XMM3
Shadow space	None	32 bytes, caller-allocated
Stack alignment	16 bytes at call	16 bytes at call
Red zone	128 bytes	None

Anything you write in assembly, any FFI shim, and any hand-rolled trampoline must be written twice or generated per-ABI.

ISA baselines. Do not assume AVX. The microarchitecture levels are the portable vocabulary:

Level	Includes	Safe assumption
`x86-64`	SSE2	Universal

Level	Includes	Safe assumption
(v1)		
`x86-64-v2`	SSE4.2, POPCNT	Safe for consumer targets today
`x86-64-v3`	AVX2, FMA, BMI	Common but **not** universal — some low-power SKUs lack it
`x86-64-v4`	AVX-512	Server/HEDT only; fragmented even there

Rule X1: compile the baseline at v2, and dispatch SIMD kernels at runtime via CPUID (or `std::simd` plus function multiversioning). Do not ship a single v3 binary and let it crash on older hardware with `SIGILL`.

6.3 ARM64 / AArch64

Increasingly the *primary* target, not the port: Apple silicon, all modern Android, AWS Graviton, and a growing share of Windows.

- ABI is **AAPCS64** — one convention, far fewer variants than x86-64
- Feature detection via `getauxval(AT_HWCAP)` on Linux/Android, `sysctlbyname` on Apple
- **NEON is baseline** on ARMv8-A. SVE/SVE2 is not — dispatch it
- Weaker memory model than x86-64. Code that is accidentally correct on x86 due to TSO **will** break here. This is the most common class of "only fails on ARM" bug, and it is almost always a missing `std::atomic` or an under-specified memory order
- **Rule X2:** run TSan on ARM64 specifically, not just on your x86 dev machine

6.4 Linux

Concern	Detail
libc	glibc vs musl are not ABI compatibl containers use musl; a glibc-built bi

Concern	Detail
	run there
glibc symbol versioning	Binaries built on new glibc fail on ol Build on the **oldest** glibc you suppor sysroot
Binary format	ELF. Understand `soname`, `RPATH`/`R` `Wl,--as-needed`
Symbol visibility	Default is "export everything," which leaks ABI. Use `-fvisibility=hid` explicit `__attribute__((visibility("d`
Distribution	AppImage / Flatpak / Snap for desk static-link or vendor for servers
Audio	PipeWire is the modern default; kee JACK paths where pro users live
Packaging	Assume nothing about installed libr Vendor your dependencies

6.5 Android

Android is Linux at Layer 3 and something quite different at Layers 2 and 4. Treat it as its own target.

Toolchain facts:

- **libc++ is the only STL** in the NDK since r18, and since r26 it ships directly from the same LLVM revision as Clang — no more version skew
- **libc is bionic**, not glibc. There is no separate `libpthread` or `librt`; that functionality is in libc and needs no explicit link. `libm` is separate but auto-linked. `libdl` must be linked explicitly
- **NDK APIs are C APIs only.** Android's C++ ABI is not guaranteed stable, so the platform exposes no C++ interfaces. This is B1 enforced by the platform vendor
- **CMake defaults to Clang's default standard.** Set `CMAKE_CXX_STANDARD` explicitly or you will silently get an old standard

- **Minimum supported OS is API 21 (Lollipop)** as of NDK r26

Target API level means something different here. In the NDK, "target API level" is the *minimum* supported API level for the app — the inverse of the SDK's meaning. Features gated on it are only available to apps whose minimum includes it.

Weak API references (fully rolled out in r26, extended to libc/libm in r28) let you call APIs newer than your `minSdkVersion` without `dlopen`/`dlsym` — but only if the *library* containing them existed at your `minSdkVersion`. Otherwise you still need the dynamic-loading fallback.

The 16 KB page size requirement — a hard Layer 4 gate.

Since **1 November 2025**, Google Play blocks new apps and updates targeting Android 15+ that do not support 16 KB

memory pages on 64-bit devices. A one-time six-month extension was available and expired **31 May 2026**. Pure Java/Kotlin apps are compliant by default; **anything shipping a `.so` is affected**, including native code pulled in transitively by third-party SDKs.

Compliance checklist:

1. **NDK r28+** and **AGP 8.5.1+** — with 16 KB-compatible prebuilt dependencies, you are compliant by default
2. Audit transitive dependencies. This is where teams get caught: a PDF renderer, a video SDK, a game engine
3. Verify in the Play Console App Bundle Explorer — the "Memory page size" field reads `Supports 16 KB` or it does not
4. Check for `PAGE_ALIGNMENT_16K` vs `PAGE_ALIGNMENT_4K` in the bundle
5. Test on a 16 KB-configured Android 15 emulator image

6. Audit your own code for hardcoded 4096-byte page assumptions in `mmap`, alignment, or custom allocators

JNI boundary rules:

#	Rule
J1	Minimize JNI transitions. They are expensive and they are not real-time safe — never on the audio path
J2	Local references are frame-scoped; global references are yours to delete. Wrap both in RAII types (P2)
J3	Attach and detach native threads explicitly, or you will leak and crash on unload
J4	Known NDK bug: `thread_local` with a non-trivial destructor segfaults if the containing library is `dlclose`d. Design around it
J5	Never let a C++ exception escape into the JVM. Catch at the JNI boundary and

#	Rule
	convert

ABIs to ship: `arm64-v8a` (mandatory), `x86_64` (emulators, Chromebooks, some tablets). `armeabi-v7a` only if your `minSdkVersion` genuinely requires 32-bit. ARMv5, MIPS, and MIPS64 have been removed for years.

6.6 Windows x64

- **MSVC ABI is not GCC/Clang ABI.** A library built with MSVC cannot expose C++ interfaces to a MinGW consumer
- **Runtime library mismatch** (`/MD` vs `/MT`, debug vs release) causes heap corruption when a pointer is allocated in one CRT and freed in another. Classic B2 violation
- `__declspec(dllexport)` / `dllimport` for symbol visibility
- clang-cl gives you Clang with MSVC ABI compatibility — useful when you want one

compiler everywhere

6.7 Other targets

- **Embedded / bare metal:** typically freestanding C++, no exceptions, no RTTI, no heap. C++17 is the common floor. RAII and `constexpr` still apply and are the highest-value features you keep
- **WebAssembly:** Emscripten, C++20-capable. Threading requires `SharedArrayBuffer` and cross-origin isolation. Audio goes through Web Audio, which imposes its own callback model
- **GPU:** CUDA (C++ dialect), SYCL/oneAPI, HIP/ROCm, Metal. C++26's `std::execution` is the standard-blessed direction for expressing this offload portably

6.8 The build and dependency layer

Tool	Role
CMake	The de facto standard. Use targets and `target_link_libraries` with visibility keywords; never global `include_directories`
CMake toolchain files	The correct mechanism for cross-compilation. Android NDK ships one
vcpkg / Conan	Dependency management. Conan Center carries the AI and audio libraries in Part 4 and Part 5
CMake Presets	Encode the platform matrix in-repo so CI and developers build identically

Minimum CI matrix for a project claiming Linux + Android + x64:

```
linux-x86_64-gcc16      (baseline, C++26)
```

```
linux-x86_64-clang      (second
opinion; different diagnostics)
linux-x86_64-asan-ubsan (P10)
linux-x86_64-tsan       (P10)
linux-aarch64           (memory
model — see X2)
android-arm64-v8a       (NDK r28+,
16 KB verified)
android-x86_64          (emulator
parity)
windows-x86_64-msvc     (if Windows
is a target at all)
```

Part 7 — Verification Toolchain

Principles are aspirations until CI enforces them.

Tool	Catches
C++26 hardened stdlib	Bounds violations

Tool	Catches
AddressSanitizer	Use-after-free, buff overflow, leaks
UndefinedBehaviorSanitizer	Signed overflow, ba shifts, misaligned access, invalid cast
ThreadSanitizer	Data races — **including on ARM6**
MemorySanitizer	Uninitialized reads (largely subsumed C++26 erroneous behavior)
Valgrind	Deeper memory analysis, no recomp
clang-tidy	Core Guidelines conformance, modernization
libFuzzer / AFL++	Parser and deserialization bug

Tool	Catches
Compiler hardening	`-D_FORTIFY_SOURCE -fstack-protector-strong -fPIE -Wl,relro,-z,now`

Sanitizers are mutually exclusive in one binary (ASan and TSan cannot coexist). Budget separate CI jobs.

Part 8 — Decision Tables

Which inference library?

If your app...	Use
...consumes a frozen model and must be portable	ONNX Runtime

If your app...	Use
...owns tensor and module semantics	LibTorch
...runs a local quantized LLM natively	llama.cpp
...is Intel-CPU-heavy	oneDNN / OpenVINO
...is small, offline, battery-sensitive	TensorFlow Lite
...needs maximum NVIDIA throughput	TensorRT-LLM / TurboMind
...orchestrates many models	Triton

Which audio layer?

If you need...	Use
Play a sound, minimum friction	miniaudio

If you need...	Use
Cross-platform plugin (VST3/AU/AAX/LV2)	JUCE
Android, lowest latency	Oboe → AAudio
Embeddable synthesis engine + a DSL	Csound 7
The DSP algorithm *is* the product	FAUST → generated C++
Portable vectorized custom kernels	`std::simd` (C++26)

Which standard?

Constraint	Target
Controlled toolchain, new project	C++26
Existing codebase	C++26 flag now, features later

Constraint	Target
Android NDK	C++20 explicit, C++26 as Clang allows
Embedded / certified	C++17 floor

Part 9 — Primary Sources

Standard and committee

- ISO C++ current status — `isocpp.org/std/status`
- cppreference C++26 feature and compiler-support tables — `cppreference.com/cpp/26`
- Herb Sutter, "C++26 is done!" trip report, March 2026
- P1000R6 — C++26 release schedule; N5046 — C++26 working revision
- P3984 (Stroustrup) — type safety profile, C++29

- P4158R0 (Hunt/Apple) — subsetting and restricting C++ for memory safety
- P3045R7 — quantities and units library

Safety

- ACM Queue, November 2025 — "Practical Security in Production: Hardening the C++ Standard Library at Massive Scale"

AI

- ONNX Runtime C++ documentation — `onnxruntime.ai/docs`
- llama.cpp / GGML — `github.com/ggml-org/llama.cpp`
- Conan Center AI library index

Audio

- JUCE — `github.com/juce-framework/JUCE`
- Csound main repository (develop branch = 7.x) — `github.com/csound/csound`

- Csound API reference — `csound.com/docs/api`
- Oboe — `github.com/google/oboe`
- miniaudio — `miniaud.io`

Platform

- Android NDK C++ library support — `developer.android.com/ndk/guides/cpp-support`
- Android NDK high-performance audio — `developer.android.com/ndk/guides/audio`
- Android 16 KB page sizes — `developer.android.com/guide/practices/p sizes`
- Android low-latency audio — `developer.android.com/games/sdk/oboe/lo latency-audio`
- System V AMD64 ABI specification; Microsoft x64 calling convention documentation

---

Appendix — Open Questions for Volume II

1. **Reflection patterns.** The feature is a decade-scale change and the idioms do not exist yet. Volume II should be written after real code accumulates.
2. `std::execution` **adoption.** Committee members acknowledge the documentation gap. A practical adapter cookbook is the highest-value follow-up.
3. **Contracts in practice.** Sustained expert dissent means the failure modes are not yet catalogued. Track this.
4. **Profiles vs. hardening.** C++29 will layer Stroustrup's profiles over C++26's hardening. The interaction is not yet designed.
5. **AI-generated C++ at scale.** The defect distribution of machine-written C++ is not

yet well characterized. Rules AI-8 through AI-11 are provisional.

6. **Csound 7 final release.** Development-branch status as of this writing; confirm the release version before pinning.